EQUIPRENT

SISTEMA DE ALQUILER DE MAQUINARIA Y MANTENIMIENTO

EVIDENCIAS MOBILE · REACT NATIVE + EXPO

GUÍA 05

Consumo de API REST y estados de pantalla

La app `equiprent-mobile` consume el backend Spring Boot (`/api/unidades-equipo`) y maneja los estados loading, success, empty y error, con Pull-to-Refresh y el detalle de la relación `UnidadEquipo 1:N Mantenimiento`.

CONTENIDO DEL INFORME

C1	**Compilación limpia**	npx tsc --noEmit sin errores
C2	**Estado Success**	Catálogo con unidades reales del backend
C3	**Pull-to-Refresh**	Spinner de RefreshControl recargando
C4	**Detalle 1:N**	Ficha de la unidad + sus mantenimientos
C5	**Estado Empty**	200 OK con arreglo vacío []
C6	**Estado Error**	Backend apagado + Reintentar conexión
QA	**Banco de preguntas**	5 preguntas de la defensa oral

ESTUDIANTE
Diana Boutier Vaca

MATERIA
Programación Aplicada 2026-2

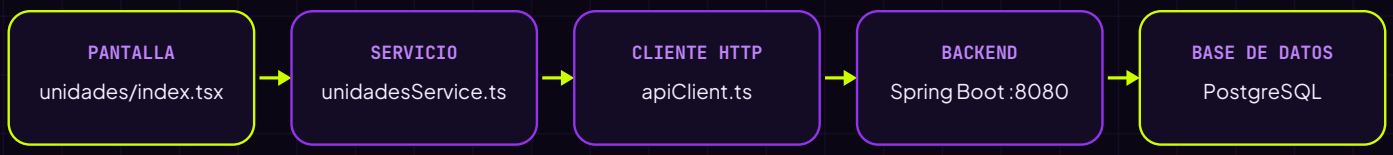
FECHA
04/11/2026

REPOSITORIO
BoutierDiana/EquipRent

01 · CONTEXTO

Cómo viaja la información

La pantalla no llama a `fetch` directamente: delega en la capa `src/services/`. Así la vista solo se ocupa del ciclo de vida, de la interfaz y de los eventos del usuario.




```
GET http://10.0.2.2:8080/api/unidades-equipo  ->  JSON  ->  setUnidades(datos)  ->  FlatList
```

Los 4 estados de la pantalla del catálogo

ESTADO	CUÁNDO OCURRE	CONTROL EN CÓDIGO	QUÉ SE DIBUJA	CAPTURA
LOADING	Promesa de cargarDatos() pendiente	`if (cargando)`	ActivityIndicator	-
SUCCESS	HTTP 200 con registros	`datos.length > 0`	FlatList de tarjetas	C2 · C3 · C4
EMPTY	HTTP 200 con arreglo []	`ListEmptyComponent`	EmptyState morado	C5
ERROR	Red caída o !respuesta.ok	`catch -> if (error)`	Mensaje + Reintentar	C6

EMPTY NO ES ERROR

Un arreglo vacío con 200 OK es un caso normal del dominio: la consulta funcionó, pero la tabla no tiene filas. El error, en cambio, es una falla de transporte (servidor apagado, conexión rechazada, 5xx) y entra por el `catch`.

CAPTURA 1 · C1

Compilación limpia de TypeScript

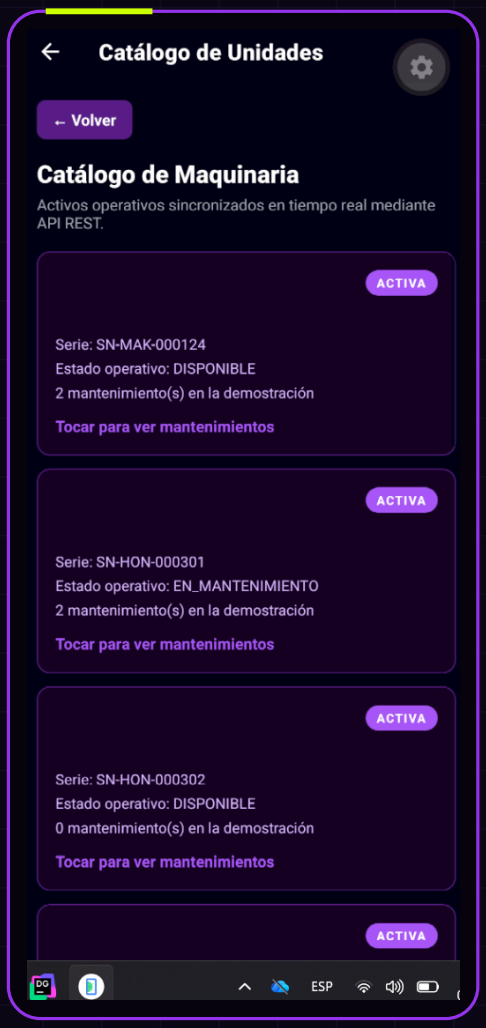
`npx tsc --noEmit` vuelve al prompt sin ninguna línea roja: código de salida 0, los tipos de servicios y pantallas son correctos.

```
PS C:\Users\ASUS\EquipRent\equiprent-mobile> npx tsc --noEmit
PS C:\Users\ASUS\EquipRent\equiprent-mobile>
```

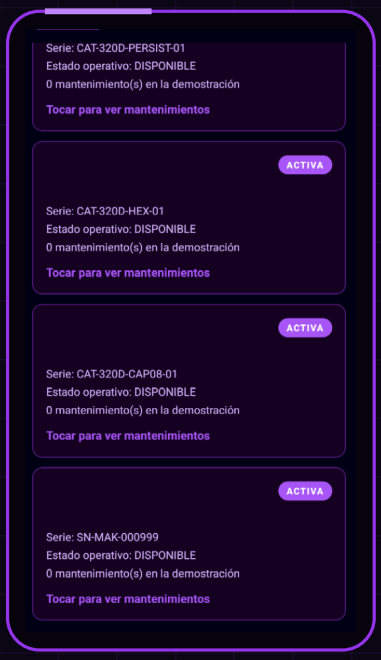
CAPTURA 2 · C2

Estado de éxito (Success)

Catálogo `/unidades` con las unidades reales devueltas por `GET /api/unidades-equipo`, con el backend encendido en IntelliJ.



C2-A · CATÁLOGO EN EL EMULADOR



C2-B · MÁS UNIDADES AL DESPLAZAR

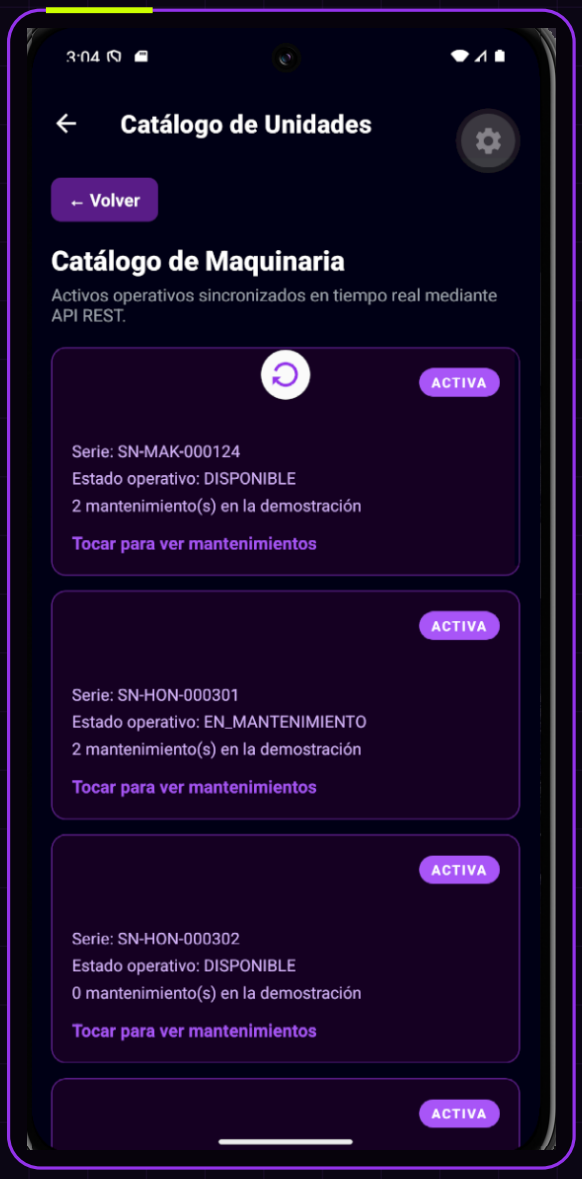
QUÉ DEMUESTRA

- Las tarjetas vienen del backend: series, estado operativo y badge ACTIVA.
- FlatList renderiza solo lo visible al desplazar la lista.

CAPTURAS 3 Y 4 · C3 · C4

Pull-to-Refresh y detalle 1:N

C3: al arrastrar la lista hacia abajo, `RefreshControl` muestra el spinner y repite el GET. C4: `/unidades/[id]` consulta `GET /api/unidades-equipo/{id}` y muestra sus mantenimientos.



C3 · SPINNER DE REFRESHCONTROL



C4 · DETALLE DE UNA UNIDAD (1:N)

FLUJO DE alRefrescar()

- setRefrescando(true) mantiene el spinner.
- unidadesService.obtenerTodas() repite el GET.
- setUnidades(datos) y setRefrescando(false) redibujan.

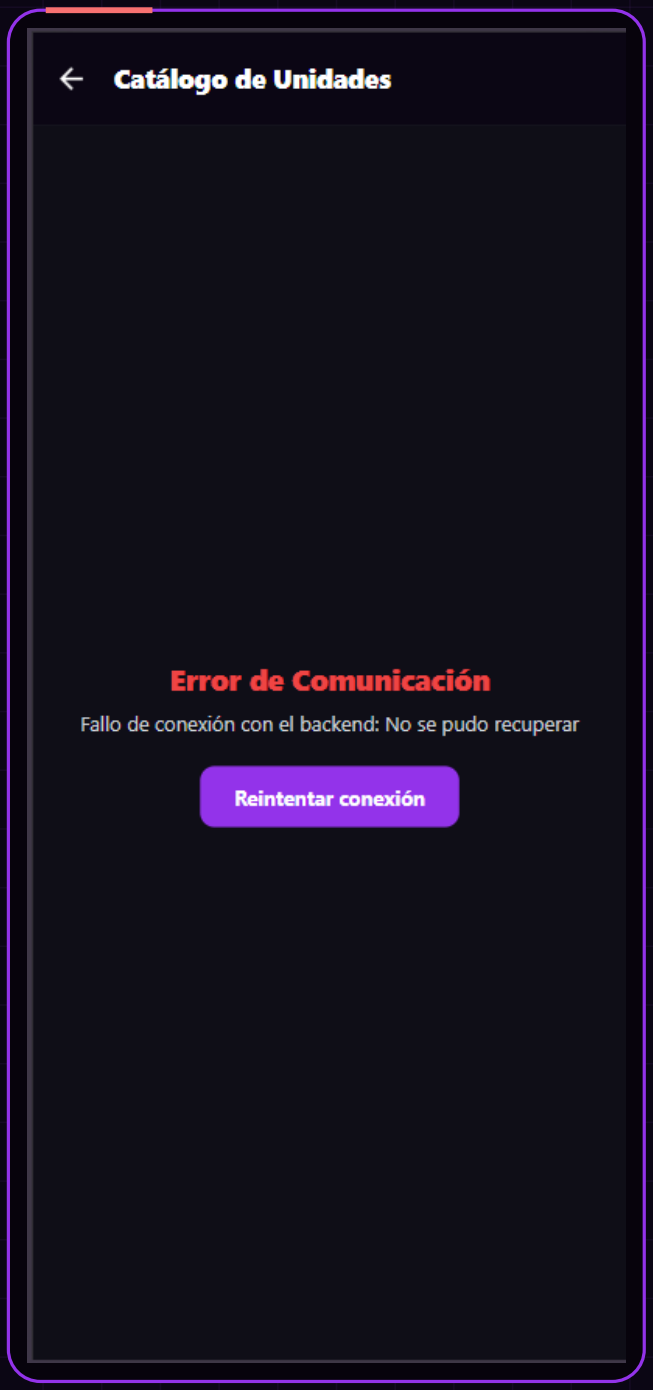
QUÉ DEMUESTRA

- El id de la ruta viaja al backend por GET /{id}.
- Una UnidadEquipo tiene N mantenimientos vinculados.

CAPTURAS 5 Y 6 · C5 · C6

Estado vacío y error de red

C5: la API responde 200 OK con `[]` y se dibuja el EmptyState. C6: con Spring Boot apagado, el `catch` guarda el mensaje en el estado `error` y aparece el botón de reintento.



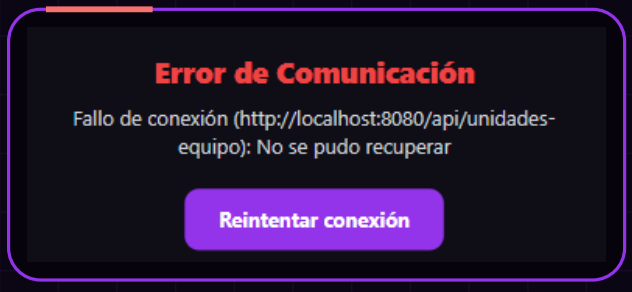
C6 · BACKEND APAGADO

C5 · EmptyState



C5 · 200 OK CON ARREGLO VACÍO

C6 · Detalle del fallo



C6 · URL Y MENSAJE DEL ERROR

DIFERENCIA CLAVE

- Empty: el servidor respondió bien, solo no hay filas.
- Error: no hubo respuesta válida; «Reintentar conexión» vuelve a llamar a cargarDatos().

DEFENSA ORAL

Banco de preguntas

01

C2 · C5 · C6

¿Cuáles son los 4 estados que maneja la pantalla del catálogo y cómo se controlan en el código?

Loading: se activa al iniciar `cargarDatos()`; mientras la promesa está pendiente se muestra un `ActivityIndicator`. Success: si la respuesta es 200 y el arreglo tiene registros, se dibujan en el `FlatList`. Empty: si la respuesta es 200 pero `datos.length === 0`, el `ListEmptyComponent` dibuja el EmptyState. Error: ante una excepción de red o `!respuesta.ok`, el `catch` guarda el mensaje en `error` y se muestra la pantalla de fallo con «Reintentar».

PRUEBA EN VIVO Abrir src/app/unidades/index.tsx y señalar if (cargando), if (error), ListEmptyComponent y el FlatList.

02

C2

¿Por qué en Android Emulator se usa 10.0.2.2 en lugar de localhost?

El emulador corre en una máquina virtual con su propia red y su propio loopback. Dentro de Android, `localhost` o `127.0.0.1` apunta al teléfono virtual, no a la computadora. `10.0.2.2` es el alias que Google reservó para llegar al localhost de la máquina anfitriona, donde corre Spring Boot.

PRUEBA EN VIVO Abrir src/services/apiClient.ts y señalar API_BASE_URL = http://10.0.2.2:8080/api.

03

C2

¿Por qué las llamadas HTTP se extraen a src/services/ y no se ejecutan con fetch directamente en la pantalla?

Por separación de responsabilidades. Las pantallas de `src/app/` solo gestionan ciclo de vida, interfaz y eventos. Centralizar el transporte en `apiClient.ts` y `unidadesService.ts` permite cambiar la URL base, agregar tokens o cambiar el cliente HTTP sin reescribir ni romper las pantallas.

PRUEBA EN VIVO Mostrar que unidades/index.tsx solo invoca unidadesService.obtenerTodas() y no conoce la URL ni las cabeceras.

DEFENSA ORAL

Banco de preguntas (cont.)

04 C5 · C6

¿Qué diferencia hay entre un error de conexión (backend caído) y un estado vacío (EmptyState)?

El error de conexión es una falla de transporte (servidor que no responde, conexión rechazada o 5xx) y entra por el `catch`. El estado vacío es un 200 OK válido: el servidor procesó la consulta pero la tabla en PostgreSQL no tiene filas. Un `[]` es un caso normal del dominio, no una excepción.

PRUEBA EN VIVO Mostrar ListEmptyComponent en el FlatList frente al bloque if (error).

05 C3

¿Cómo viaja la información desde el Pull-to-Refresh hasta que se dibuja el resultado?

1) El usuario arrastra y `RefreshControl` ejecuta `alRefrescar()`. 2) `refrescando = true` mantiene el spinner. 3) `unidadesService.obtenerTodas()` envía un GET por `requestHttp` a `http://10.0.2.2:8080/api/unidades-equipo`. 4) Spring Boot consulta el repositorio JPA, lee `unidades_equipo` en PostgreSQL y responde JSON. 5) `setUnidades(datos)` actualiza el estado y `setRefrescando(false)` oculta el spinner.

PRUEBA EN VIVO Explicar alRefrescar() y el ciclo asíncrono en src/app/unidades/index.tsx.

CONCLUSIÓN

Resultado de la Guía 05

C1	Compilación limpia (tsc)	CUMPLE
C2	Estado Success	CUMPLE
C3	Pull-to-Refresh	CUMPLE
C4	Detalle 1:N	CUMPLE
C5	Estado Empty	CUMPLE
C6	Estado Error + Reintentar	CUMPLE

Ideas clave para la defensa

- async/await + response.ok: fetch solo rechaza la promesa si falla la red; un 404 o 500 hay que detectarlo revisando response.ok y lanzando el error a mano.
- FlatList en vez de .map(): virtualiza la lista (renderiza solo lo visible) y trae integrados ListEmptyComponent y RefreshControl.
- Capa de servicios: la pantalla pide datos; el servicio sabe a qué URL y con qué cabeceras.
- Relación 1:N: el detalle de una UnidadEquipo muestra las N órdenes de Mantenimiento que le pertenecen.